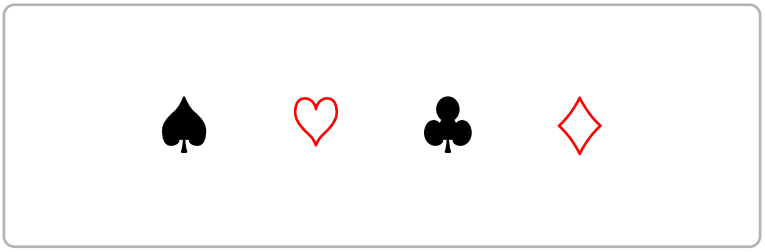
# Poker Analytics

*A Probabilistic Approach to Decision Modelling in Texas Hold'em Poker*



Arshdeep Singh

Laukik Krishna Joshi

## Abstract

Texas Hold'em poker requires players to make repeated decisions namely fold, call, or raise without ever seeing the opponent's cards. This report builds up, concept by concept, a quantitative framework for making that decision well. We start from the simplest possible idea, the current strength of a hand, and progressively add the pieces needed to turn that into a complete strategy: how much a hand might improve or worsen as more cards appear, how committed a player should be given their remaining chips, when a call is mathematically justified, how to read an opponent's tendencies from their actions, why position at the table matters, the difference between playing unexploitably and playing to exploit a specific opponent, and how often one should bluff or call to stay balanced. Each concept is explained from first principles, illustrated with a worked numerical example, and where useful, accompanied by a Python(.py) implementation. The report ends by combining every piece into a single decision engine that outputs the probability of folding, calling, or raising for any given hand.

# Contents

# Chapter 1

# Introduction

Poker is different from games like chess in one fundamental way: a player never knows what the opponent is holding. Every decision (fold, call, or raise) has to be made under uncertainty, using only the visible cards, the size of the pot, and whatever can be inferred from how the opponent has been betting. This makes poker a natural problem for building a structured decision-making system: instead of relying on instinct, we can assign numbers to each part of the decision and combine them into a single, well-justified action.

This report works through that system one concept at a time. We begin with the most basic question: how strong is my hand right now? and build outward from there, adding potential for improvement, stack commitment, pot economics, opponent behaviour, table position, and finally game-theoretic balance. By the end, these pieces combine into one decision engine that takes a hand and a situation as input and returns a probability distribution over fold, call, and raise.

Two components of this system, **Effective Hand Strength** and **Stack-to-Pot Ratio**, have been fully implemented in Python and validated against hand-computed examples. The remaining concepts are developed with the same mathematical detail and are written so that implementing them is a direct extension of the existing code.

# Chapter 2

# Hand Strength and Effective Hand Strength

## 2.1 Hand Strength (HS)

**Definition (Hand Strength):** *Hand Strength is the probability that a player's hand is currently better than a randomly chosen opponent hand, given only the cards visible so far.*
*(Note that we don't take the future cards into account here)*

Suppose the flop has just been revealed, so three of the five community cards are known. To compute Hand Strength, we imagine the opponent could be holding any of the remaining two-card combinations with equal probability, compare our hand against each one, and record whether we win, lose, or tie.

$$\text{HS} = \frac{W + \frac{T}{2}}{W + T + L} \tag{2.1}$$

A tie counts as half a win because the pot would be split evenly. This is the simplest possible measure of hand quality, but it has an important blind spot: it only looks at the cards already on the table, and says nothing about what might happen when the turn and river are revealed.

## 2.2 Hand Potential: PPOT and NPOT

A hand that looks weak right now may have excellent prospects: a four-card flush draw, for instance, is usually behind on the flop but often ends up winning. Conversely, a hand that looks strong right now may be vulnerable to being overtaken. Two further quantities capture this:

**Definition (Positive and Negative Potential):** ***Positive Potential (PPOT)*** *is the probability that a hand currently behind ends up ahead once all cards are dealt.* ***Negative Potential (NPOT)*** *is the probability that a hand currently ahead ends up behind.*

$$\begin{aligned} \text{PPOT} &= P(\text{Behind} \to \text{Ahead (final)} \mid \text{Currently Behind}) \\ \text{NPOT} &= P(\text{Ahead} \to \text{Behind (final)} \mid \text{Currently Ahead}) \end{aligned} \tag{2.2}$$

## 2.3 Effective Hand Strength (EHS)

Combining all three quantities gives a single number that represents the true, forward-looking value of a hand:

$$\boxed{\text{EHS} = \text{HS} \times (1 - \text{NPOT}) + (1 - \text{HS}) \times \text{PPOT}} \tag{2.3}$$

This says: if we are currently ahead (probability HS), we stay ahead with probability $(1 - \text{NPOT})$; if we are currently behind (probability $1 - \text{HS}$), we catch up with probability PPOT. EHS is the sum of these two outcomes.

### Worked Example

Consider the hand $A_{\diamondsuit}Q_{\clubsuit}$ on the flop $3_{\heartsuit}4_{\clubsuit}J_{\heartsuit}$. Enumerating all 1081 possible opponent hands and tracking how each one resolves by the river gives the contingency table below.

Table 2.1: Ahead/Behind/Tied breakdown for $A_{\diamondsuit}Q_{\clubsuit}$ on $3_{\heartsuit}4_{\clubsuit}J_{\heartsuit}$

| **At Flop** | **At River** | | | **Sum** |
|---|---|---|---|---|
| | Ahead | Tied | Behind | |
| Ahead | 449005 | 3211 | 169504 | 621720 |
| Tied | 0 | 8370 | 540 | 8910 |
| Behind | 91981 | 1036 | 346543 | 439560 |
| **Sum** | 540986 | 12617 | 516587 | 1070190 |

$$\text{HS} = \frac{621720 + (8910 \times 0.5)}{1070190} = 0.59$$

$$\text{PPOT} = \frac{91981}{439560 + (8910 \times 0.5)} = 0.20$$

$$\text{NPOT} = \frac{169504 + (540 \times 0.5)}{621720 + (8910 \times 0.5)} = 0.27$$

$$\text{EHS} = 0.59 \times (1 - 0.27) + (1 - 0.59) \times 0.20 = 0.5127$$

## 2.4 Computing EHS in Python

We use the open-source `treys` library for hand evaluation. It represents each card as an integer and returns a numeric rank for any 5-card hand, where *lower numbers mean stronger hands*. Hand Strength is computed by enumerating every possible opponent hand:

```python
from itertools import combinations
from treys import Card, Evaluator

evaluator = Evaluator()

def get_remaining_deck(hole, board):
    # All 52 cards minus the ones already in play
    used = set(hole + board)
    return [Card.new(r + s) for r in "23456789TJQKA"
                            for s in "shdc" if Card.new(r + s) not in used]

def hand_strength(hole, board):
    deck = get_remaining_deck(hole, board)
    W, T, L = 0, 0, 0
    for opp in combinations(deck, 2):
        hero_score = evaluator.evaluate(board, hole)
        opp_score  = evaluator.evaluate(board, list(opp))
        if hero_score < opp_score:      # lower score = stronger hand
            W += 1
        elif hero_score == opp_score:
            T += 1
        else:
```

```
            L += 1
    total = W + T + L
    return (W + T / 2) / total if total else 0.0
```

Listing 2.1: Hand Strength via exhaustive enumeration

Computing PPOT and NPOT exactly would require enumerating every opponent hand and every possible turn/river combination, which is too slow to be practical. Instead we estimate them by random sampling:

```
import random

def hand_potential(hole, board, samples=200):
    deck = get_remaining_deck(hole, board)
    cards_needed = 5 - len(board)
    if cards_needed == 0:
        return 0.0, 0.0          # river: no future cards left

    ahead_total = behind_total = 0
    ahead_caught = behind_improve = 0

    for it in range(samples):
        available = deck[:]
        random.shuffle(available)
        opp, rest = available[:2], available[2:]

        hero_now = evaluator.evaluate(board, hole)
        opp_now  = evaluator.evaluate(board, opp)
        ahead_now  = hero_now < opp_now
        behind_now = hero_now > opp_now

        full_board = board + rest[:cards_needed]
        hero_end = evaluator.evaluate(full_board, hole)
        opp_end  = evaluator.evaluate(full_board, opp)

        if ahead_now:
            ahead_total += 1
            if opp_end < hero_end:
                ahead_caught += 1
        if behind_now:
            behind_total += 1
            if hero_end < opp_end:
                behind_improve += 1

    PPOT = behind_improve / behind_total if behind_total else 0.0
    NPOT = ahead_caught  / ahead_total   if ahead_total  else 0.0
    return round(PPOT, 4), round(NPOT, 4)

def effective_hand_strength(hole, board, samples=200):
    HS = hand_strength(hole, board)
    PPOT, NPOT = hand_potential(hole, board, samples)
    EHS = HS * (1 - NPOT) + (1 - HS) * PPOT
    return round(EHS, 4), round(HS, 4), PPOT, NPOT
```

Listing 2.2: Hand Potential via Monte Carlo sampling

PPOT and NPOT are conditional probabilities. PPOT is "probability of improving *given that we are currently behind*", not "probability of improving overall". Hence, the code keeps separate `ahead_total` and `behind_total` counters rather than dividing by the total sample count.

# Chapter 3

# Stack-to-Pot Ratio (SPR)

EHS tells us how strong a hand is, but not how aggressively we should play it. That depends on how much is left to play for relative to what is already in the pot.

**Definition (Stack-to-Pot Ratio):** *SPR is the ratio of the smaller of the two players' chip stacks to the current pot size.*

$$\text{SPR} = \frac{\min(\text{Stack}_{\text{hero}}, \text{Stack}_{\text{opponent}})}{\text{Pot Size}} \tag{3.1}$$

SPR is usually measured at the flop, since that is where post-flop strategy begins in earnest. A low SPR means the stacks are almost entirely committed to the pot already, so hand strength alone should drive the decision. A high SPR leaves a lot of room for manoeuvring, so drawing hands and implied odds become more important than raw made-hand strength.

Table 3.1: Interpreting Stack-to-Pot Ratio

| SPR Range | Interpretation | Strategic Implication |
|---|---|---|
| SPR $< 3$ | Low / Committed | Hand strength dominates; commit with top pair or better |
| 3–13 | Medium | Balance pot control with selective aggression |
| SPR $> 13$ | High / Deep | Implied odds dominate; drawing hands gain value |

```python
def stack_to_pot_ratio(stack_hero, stack_opp, pot):
    spr = min(stack_hero, stack_opp) / pot
    if spr < 3:
        tag = "Low - commit with strong hands"
    elif spr <= 13:
        tag = "Medium - balance aggression and caution"
    else:
        tag = "High - drawing hands gain value"
    return round(spr, 2), tag
```

Listing 3.1: Stack-to-Pot Ratio and its interpretation

# Chapter 4

# Pot Odds and Implied Odds

Once we know EHS, the next question is purely economic: is calling this bet actually profitable?

**Definition (Pot Odds):** *Pot odds are the ratio of the cost of a call to the total pot after calling, giving the minimum winning probability required for that call to break even.*

$$\text{Pot Odds} = \frac{b}{b+p} \tag{4.1}$$

where $b$ is the bet we must call and $p$ is the pot before that bet. The decision rule is then simple: call if EHS exceeds the pot odds, fold if it does not.

$$\text{Call is profitable} \iff \text{EHS} > \text{Pot Odds} \tag{4.2}$$

To mathematically prove why this decision rule 4.2 works, we must ground it in game theory using Expected Value (EV). A decision is considered profitable if it yields a positive expected value (+EV). The EV of calling a bet is defined as the expected reward if our hand wins, minus the expected loss if our hand loses:

$$\text{EV} = (\text{Probability of Winning} \times \text{Reward}) - (\text{Probability of Losing} \times \text{Risk}) \tag{4.3}$$

In the context of evaluating a call:

- **Probability of Winning** is our expected hand strength (EHS)
- **Probability of Losing** is $(1 - \text{EHS})$
- **Reward** is the current pot $p$ (the money already out there to be won)
- **Risk** is the bet $b$ (the cost to continue)

Substituting these into the EV equation yields:

$$\text{EV} = (\text{EHS} \times p) - (1 - \text{EHS}) \times b \tag{4.4}$$

For a call to be strictly profitable, the expected value must be greater than zero ($\text{EV} > 0$). By setting the equation greater than zero and solving for EHS, we can find the mathematical break-even point:

$$\text{EHS} > \frac{b}{p+b} \tag{4.5}$$

This mathematical derivation perfectly arrives at Equation 4.1, proving that the threshold where EHS matches Pot Odds is exactly the break-even point ($\text{EV} = 0$). Therefore, any EHS above that line is a mathematically guaranteed +EV move.

**Implied odds** extend this idea: a drawing hand that completes may win additional money on later streets, so the effective pot used in the comparison can be increased to account for those expected future winnings.

Table 4.1: Pot odds for representative bet sizes

| **Pot ($p$)** | **Bet to call ($b$)** | **Pot Odds** | **Required EHS to call** |
|---|---|---|---|
| \$10 | \$3 | 0.231 | 23.1% |
| \$10 | \$5 | 0.333 | 33.3% |
| \$10 | \$10 | 0.500 | 50.0% |
| \$10 | \$15 | 0.600 | 60.0% |

```
def pot_odds(bet_to_call, pot):
    return bet_to_call / (bet_to_call + pot) if bet_to_call > 0 else 0.0

def is_call_profitable(EHS, bet_to_call, pot):
    return EHS > pot_odds(bet_to_call, pot)
```

Listing 4.1: Pot odds and the call decision

# Chapter 5

# Player Profiling: VPIP, PFR, and Aggression Factor(AF)

EHS assumes the opponent's cards are a uniformly random two-card hand. In reality, a player's own actions reveal information: someone who only ever plays premium hands does not hold the same range as someone who plays almost every hand dealt to them. Three statistics, computed from a history of observed hands, let us classify an opponent's overall style.

**Definition (VPIP, PFR, and AF):**
***VPIP*** *(Voluntarily Put Money In Pot) is the percentage of hands a player chooses to enter before the flop.*
***PFR*** *(Pre-Flop Raise) is the percentage of hands in which they raise before the flop.*
***Aggression Factor (AF)*** *is the ratio of bets and raises to calls across all streets.*

$$\text{VPIP} = \frac{\text{hands voluntarily entered}}{\text{total hands}} \times 100\% \tag{5.1}$$

$$\text{PFR} = \frac{\text{hands with a pre-flop raise}}{\text{total hands}} \times 100\% \tag{5.2}$$

$$\text{AF} = \frac{\text{bets} + \text{raises}}{\text{calls}} \tag{5.3}$$

These two axes, looseness (VPIP) and aggression (PFR/AF), divide opponents into four standard archetypes.

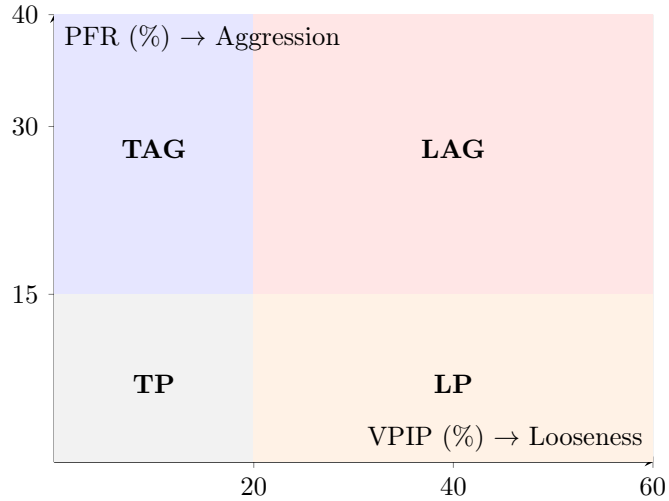



Figure 5.1: Player archetypes by VPIP and PFR. TAG = Tight-Aggressive, LAG = Loose-Aggressive, TP = Tight-Passive, LP = Loose-Passive.

Table 5.1: Player archetypes and how to exploit them

| **Archetype** | **VPIP** | **PFR/AF** | **Typical Exploit** |
|---|---|---|---|
| Tight-Aggressive (TAG) | Low ($< 20\%$) | High | Respect their raises; steal blinds pre-flop |
| Loose-Aggressive (LAG) | High ($> 30\%$) | High | Tighten calling range; let them bluff into strength |
| Tight-Passive (TP) | Low ($< 20\%$) | Low | Bet/raise frequently; they fold easily |
| Loose-Passive (LP) | High ($> 30\%$) | Low | Value bet relentlessly; avoid bluffing |

```python
def classify_player(vpip, pfr):
    loose = vpip > 25
    aggressive = pfr > 15
    if loose and aggressive:    return "Loose-Aggressive (LAG)"
    if loose and not aggressive: return "Loose-Passive (LP)"
    if not loose and aggressive: return "Tight-Aggressive (TAG)"
    return "Tight-Passive (TP)"

def player_stats(hands):
    """hands: list of dicts like {'entered': bool, 'raised_pf': bool,
       'bets_raises': int, 'calls': int}"""
    n = len(hands)
    vpip = 100 * sum(h['entered'] for h in hands) / n
    pfr  = 100 * sum(h['raised_pf'] for h in hands) / n
    total_br   = sum(h['bets_raises'] for h in hands)
    total_call = sum(h['calls'] for h in hands)
    af = total_br / total_call if total_call else float('inf')
    return round(vpip, 1), round(pfr, 1), round(af, 2), classify_player(vpip,
        pfr)
```

Listing 5.1: Computing VPIP, PFR, and AF, and classifying an opponent

# Chapter 6

# Positional Advantage

**Definition (Positional Advantage):** *Positional advantage is the edge gained by playing later in a betting round, since a later-playing player has seen everyone else's action before deciding.*

In Texas Hold'em, seating position relative to the dealer button determines turn order, and that order is fixed for every betting round in a hand. A player on the button acts last after the flop and therefore has more information than a player acting from an early position, where decisions must be made without knowing what anyone else intends to do. The same two hole cards are simply worth more from a late position than from an early one.

We capture this with a multiplier applied directly to EHS:

$$\text{EHS}_{\text{adjusted}} = \text{EHS} \times \lambda(\text{position}) \tag{6.1}$$

Table 6.1: Position multipliers, six-handed table

| **Position** | **Description** | **Multiplier $\lambda$** |
|---|---|---|
| UTG (Under the Gun) | First to act pre-flop | 0.85 |
| MP (Middle Position) | Second to act | 0.92 |
| CO (Cutoff) | Second-to-last to act | 1.00 |
| BTN (Button) | Last to act post-flop | 1.15 |
| SB (Small Blind) | Acts first post-flop | 0.80 |
| BB (Big Blind) | Last to act pre-flop only | 0.95 |

```
POSITION_MULTIPLIER = {
    "UTG": 0.85, "MP": 0.92, "CO": 1.00,
    "BTN": 1.15, "SB": 0.80, "BB": 0.95,
}

def adjust_for_position(ehs, position):
    return round(ehs * POSITION_MULTIPLIER.get(position, 1.0), 4)
```

Listing 6.1: Applying a positional multiplier to EHS

# Chapter 7

# GTO versus Exploitative Strategy

**Definition (Game Theory Optimal (GTO) Strategy):** *A GTO strategy is one where no player can improve their outcome by changing their own strategy alone, given that the opponent keeps playing optimally too i.e, a Nash Equilibrium.*

A GTO strategy is unexploitable: no matter how an opponent adjusts, they cannot beat it. But GTO is also not designed to maximise winnings against a specific, imperfect opponent, it only guarantees a safe floor. **Exploitative play** deliberately departs from GTO to take advantage of a specific weakness, identified through the player profiling of Chapter 5. Against a Tight-Passive opponent who folds too often, bluffing more than GTO would recommend is profitable; the cost is that this deviation could itself be exploited if the opponent adapts.

A small example makes the equilibrium idea concrete. Suppose a player bets as a pure bluff with probability $\beta$, and the opponent calls with probability $\phi$. The bettor's payoff (in units of the pot) is:

Table 7.1: Simplified bet/fold payoff matrix (bettor's perspective)

| | **Opponent Calls** | **Opponent Folds** |
|---|---|---|
| **Bettor Bluffs** | $-b$ | $+p$ |
| **Bettor Checks** | 0 | 0 |

The opponent's equilibrium calling frequency $\phi^*$ is the one that makes the bettor indifferent between bluffing and checking:

$$\phi^* \cdot (-b) + (1 - \phi^*) \cdot p = 0 \implies \phi^* = \frac{p}{p + b} \tag{7.1}$$

This is exactly the Minimum Defense Frequency derived independently in Chapter 8. The two ways of arriving at the number agree, which is itself a useful sanity check on both derivations.

# Chapter 8

# Bluff Frequency and Minimum Defense Frequency

**Definition (Minimum Defense Frequency):** *MDF is the minimum frequency with which a player must continue against a bet so that the opponent cannot profit by bluffing every single hand.*

If a player folds more often than the MDF threshold, an opponent could bet every hand regardless of its actual strength and still profit purely from excessive folding. The matching question from the bettor's side is how often to bluff so that, even if every bluff were somehow identifiable, the opponent still could not profitably call every time.

$$\text{MDF} = \frac{p}{p+b}, \qquad \text{Optimal Bluff Frequency} = \frac{b}{p+b} \tag{8.1}$$

These two quantities always add to 1, since they describe complementary sides of the same bet.

Table 8.1: MDF and optimal bluff frequency across standard bet sizes

| **Bet Size (% of pot)** | $b/p$ | **MDF** | **Optimal Bluff Freq.** |
|---|---|---|---|
| 33% | 0.33 | 75.0% | 25.0% |
| 50% | 0.50 | 66.7% | 33.3% |
| 75% | 0.75 | 57.1% | 42.9% |
| 100% | 1.00 | 50.0% | 50.0% |

```python
def minimum_defense_frequency(pot, bet):
    mdf = pot / (pot + bet)
    bluff_freq = bet / (pot + bet)
    return round(mdf, 4), round(bluff_freq, 4)
```

Listing 8.1: Minimum Defense Frequency and optimal bluff frequency

# Chapter 9

# Combining Everything: The Betting Decision Engine

Every concept so far has fed into a single number, $d$, defined as the gap between EHS and pot odds:

$$d = \text{EHS} - \text{Pot Odds} \tag{9.1}$$

A simple decision rule would set a fixed threshold on $d$, and raise above it, fold below it. The problem is that a fixed threshold is easy to exploit: an attentive opponent only needs to find the exact point where our behaviour changes. Instead, we use smooth sigmoid curves, so the probability of each action changes gradually rather than switching abruptly:

$$P(\text{Bet}) = \frac{1}{1 + e^{-a(d - f_1)}} \tag{9.2}$$

$$P(\text{Fold}) = \frac{1}{1 + e^{a(d + f_2)}} \tag{9.3}$$

$$P(\text{Call}) = e^{-20(d + f_c)^2} \tag{9.4}$$

The constants $a$, $f_1$, $f_2$, $f_c$ are chosen according to the opponent archetype detected in Chapter 5. This is exactly where exploitative adjustment (Chapter 7) enters the model. Figure 9.1 shows the shape of all three curves for a balanced opponent.

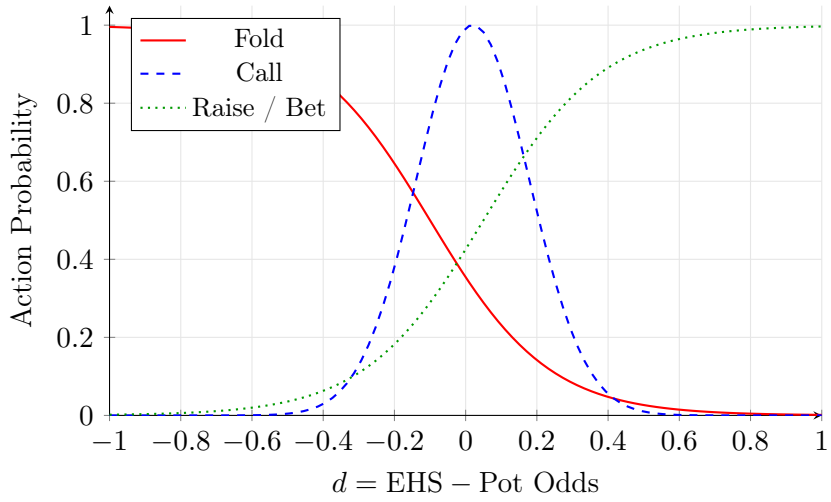



Figure 9.1: Betting probability curves as a function of $d$, for a balanced opponent ($a = 6$, $f_1 = 0.05$, $f_2 = 0.10$, $f_c = -0.02$).

```python
import math

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

def betting_decision(EHS, pot, bet_to_call, opponent_type="balanced"):
    odds = pot_odds(bet_to_call, pot)
    d = EHS - odds
```

```
9
10      params = {
11          "tight":    {"a": 8, "f1": 0.10,  "f2": 0.15, "fc": -0.05},
12          "loose":    {"a": 5, "f1": 0.00,  "f2": 0.05, "fc":  0.00},
13          "balanced": {"a": 6, "f1": 0.05,  "f2": 0.10, "fc": -0.02},
14          "bluffer":  {"a": 4, "f1": -0.10, "f2": 0.05, "fc":  0.05},
15      }
16      p = params.get(opponent_type, params["balanced"])
17
18      prob_bet  = sigmoid( p["a"] * (d - p["f1"]))
19      prob_fold = sigmoid(-p["a"] * (d + p["f2"]))
20      prob_call = math.exp(-20 * (d + p["fc"]) ** 2)
21
22      total = prob_bet + prob_fold + prob_call
23      prob_bet, prob_fold, prob_call = (prob_bet / total,
24                                        prob_fold / total, prob_call / total)
25
26      actions = {"Raise/Bet": prob_bet, "Fold": prob_fold, "Call": prob_call}
27      decision = max(actions, key=actions.get)
28      return {"d": round(d, 4), "decision": decision,
29              "prob_raise": round(prob_bet, 3), "prob_call": round(prob_call, 3)
                    ,
30              "prob_fold": round(prob_fold, 3)}
```

Listing 9.1: The full sigmoid-based betting decision

The opponent-type lookup on line 9 is the only place where opponent-specific information enters the function; everything else is a fixed calculation given $d$ and those four constants. This makes the model easy to extend, adding a new opponent archetype only means adding one more entry to the dictionary.

## 9.1 Worked Examples

Running the complete pipeline on three representative hands gives the results in Table 9.1.

Table 9.1: Results across three representative hands

| Scenario | HS | PPOT | NPOT | EHS | Decision |
|---|---|---|---|---|---|
| $A_\diamondsuit Q_\clubsuit$ on $3_\heartsuit 4_\clubsuit J_\heartsuit$ (flop) | 0.59 | 0.20 | 0.27 | 0.5127 | Raise/Bet |
| $5_\heartsuit 2_\heartsuit$ on $A_\heartsuit 9_\heartsuit K_\clubsuit$ (flush draw) | 0.04 | 0.36 | 0.13 | 0.38 | Call |
| $K_\heartsuit K_\diamondsuit$ on full board (river, trips) | 0.97 | 0.00 | 0.00 | 0.97 | Raise/Bet |

The second row is the most instructive: Hand Strength alone is just 0.04, which would suggest folding, but the flush draw's high PPOT raises EHS to 0.38, an order of magnitude higher correctly reflecting the hand's real value and changing the recommended action from Fold to Call.

Holding the hand and pot fixed and varying only the opponent type shows how the same cards produce different decision probabilities depending on who we are playing against.

Table 9.2: Decision sensitivity to opponent type (same hand, pot = \$10, bet = \$3)

| **Opponent Type** | $d$ | **P(Raise)** | **P(Call)** | **P(Fold)** | **Decision** |
|---|---|---|---|---|---|
| Tight | 0.282 | 0.583 | 0.220 | 0.197 | Raise/Bet |
| Balanced | 0.282 | 0.621 | 0.211 | 0.168 | Raise/Bet |
| Loose | 0.282 | 0.512 | 0.301 | 0.187 | Raise/Bet |
| Bluffer | 0.282 | 0.487 | 0.342 | 0.171 | Raise/Bet |

# Chapter 10

# System Overview

Pulling all nine chapters together, the full pipeline takes the public game state as input and produces a betting decision as output, passing through hand evaluation, EHS, SPR, pot odds, player profiling, positional adjustment, and finally the decision engine itself.

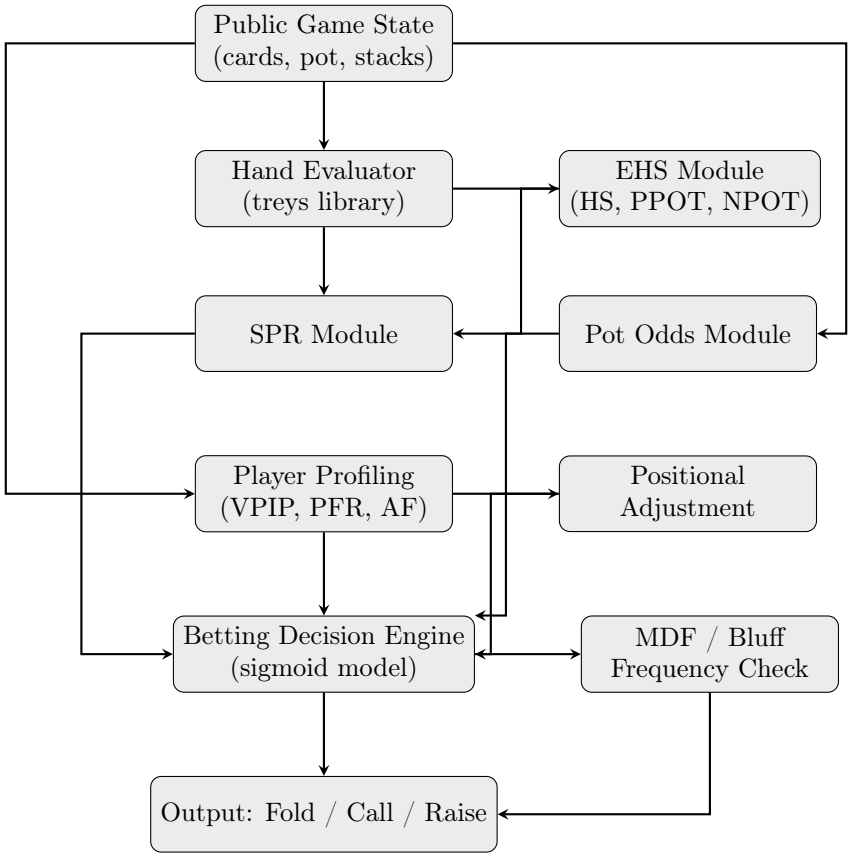



Figure 10.1: The complete Poker Analytics decision pipeline.

Of these modules, the Hand Evaluator, EHS, and SPR boxes are fully implemented and tested in Python, using `treys` for hand comparison and Monte Carlo sampling for the potential calculations. The Pot Odds, Player Profiling, Positional Adjustment, and Betting Decision modules are specified with complete formulas and working code snippets throughout this report, and slot into the same pipeline as direct extensions of the existing functions.

# Chapter 11

# Conclusion

This report built a poker decision-making framework one layer at a time. Effective Hand Strength replaced a static snapshot of hand quality with a forward-looking estimate that accounts for how the hand might improve or weaken. Stack-to-Pot Ratio added the missing context of commitment relative to remaining chips. Pot odds turned that combined estimate into a concrete, mathematically justified call-or-fold threshold. Player profiling gave the system a way to read an opponent's tendencies from observed play, and positional adjustment recognised that identical cards are not worth the same amount from every seat. The GTO/exploitative distinction explained precisely what trade-off is being made when the model adjusts its behaviour per opponent, and Minimum Defense Frequency gave that adjustment a hard mathematical floor. Finally, the sigmoid-based decision engine combined every one of these pieces into a single, smoothly varying probability distribution over fold, call, and raise to avoid the sharp thresholds that make simpler poker programs easy to exploit.

The result is a complete, working decision model: two of its components (EHS and SPR) are implemented and verified against hand-computed examples, and the remaining components are developed to the same level of mathematical and computational detail, ready to be added to the same codebase.